

Technische Dokumentation: Meine Favoriten

1. Projektübersicht

„Meine Favoriten“ ist eine reine Client-side Single Page Application (SPA) zur Verwaltung von Lesezeichen. Die Anwendung verzichtet vollständig auf ein Backend und speichert alle Daten lokal im Browser des Nutzers. Der Fokus liegt auf Datenschutz, lokaler Datensouveränität und geringer Ressourcennutzung.

Die Architektur basiert auf einer statischen HTML/CSS/JS-Struktur ohne externe Abhängigkeiten (Vanilla JS). Es werden keine Bibliotheken oder Frameworks verwendet. Ein besonderes Merkmal ist der datenschutzkonforme Ansatz: Es werden keine externen Dienste (wie die Google Favicon API) für die Darstellung von Icons genutzt, und es kommen keine Tracking-Codes zum Einsatz. Die Datenpersistenz erfolgt ausschließlich über die `localStorage` API des Browsers.

2. Datenmodell & Speicherung

Die Anwendung verwaltet drei primäre Datensätze im `localStorage`. Alle Daten werden als JSON-Strings serialisiert gespeichert.

Schlüsselkonstanten

Im Quellcode sind drei eindeutige Schlüssel definiert, um die Datenbereiche zu trennen:

- `favorite-links-v1` : Speichert den Hauptdatensatz aller Favoriten-Links.
- `favorite-category-order-v1` : Speichert die vom Nutzer definierte Reihenfolge der Kategorien.
- `favorite-visit-counts-v1` : Speichert die Nutzungsstatistik (Besuchszähler) pro Link.

Datenstrukturen

Links (`favorite-links-v1`) Dieser Datensatz ist ein Array von Objekten. Jeder Link besitzt eine eindeutige UUID, um ihn auch bei Namensgleichheit sicher identifizieren zu können.

```
[
  {
    "id": "uuid-v4-string",
    "title": "Beispiel Titel",
    "url": "https://example.com",
    "category": "Kategorie Name"
  }
]
```

Kategorien-Reihenfolge (`favorite-category-order-v1`) Ein einfaches Array von Strings, das die explizite Sortierreihenfolge der Kategorien widerspiegelt, wie sie der Nutzer im Sortierdialog festgelegt hat.

```
["Nachrichten", "Wissen", "Allgemein"]
```

Besuchszähler (`favorite-visit-counts-v1`) Ein Objekt (als Map genutzt), das die Klickhäufigkeit pro Link-ID speichert. Dies dient der Generierung der automatischen Kategorie „Am häufigsten besucht“.

```
{
  "uuid-v4-string": 42,
  "another-uuid": 5
}
```

Speichermanagement

Da der `localStorage` eines Browsers typischerweise auf 5 bis 10 MB begrenzt ist, implementiert die Funktion `safeLocalStorageSetItem` eine robuste Fehlerbehandlung. Beim Versuch, Daten zu speichern, wird ein `try-catch` -Block verwendet. Tritt ein `QuotaExceededError` auf, initiiert das System eine automatische Bereinigung: Es werden verwaiste Einträge in den Besuchszählern (`visitCounts`) gelöscht, also solche IDs, die nicht mehr in der aktuellen Link-Liste existieren. Gelingt das Speichern danach immer noch nicht, wird dem Nutzer eine kritische Warnmeldung angezeigt mit der Aufforderung, einen manuellen Export durchzuführen und alte Daten zu löschen.

3. Kernlogik & Funktionen

Initialisierung

Beim Laden der Seite (`DOMContentLoaded`) erfolgt die Initialisierung in mehreren Schritten. Zuerst werden die Daten aus dem `localStorage` gelesen (`loadLinks` , `loadCategoryOrder` , `loadVisitCounts`). Falls keine Daten vorhanden sind oder ein Lesefehler auftritt, werden Default-Werte (z. B. die Links zu Tagesschau und Wikipedia) gesetzt. Anschließend wird die `update()` -Funktion aufgerufen, welche die Kategorien synchronisiert, verwaiste Zählraten bereinigt und das vollständige Rendering der Oberfläche auslöst.

URL- und Kategorien-Normalisierung

Um Datenkonsistenz zu gewährleisten, werden Eingaben vor der Speicherung normalisiert. Die Funktion `normalizeUrl` trimmt Leerzeichen und prüft mittels regulärem Ausdruck, ob ein Protokoll vorhanden ist. Fehlt dieses, wird automatisch `https://` vorangestellt. Spezielle Protokolle wie `obsidian://` oder `msteams:` werden erkannt und beibehalten. Die Funktion `normalizeCategory` stellt sicher, dass Kategorienamen getrimmt werden und fällt bei leerer Eingabe auf den Standardwert „Allgemein“ zurück.

Favoriten-Logik

Beim Hinzufügen eines neuen Links wird diesem eine eindeutige UUID via `crypto.randomUUID()` zugewiesen. Neue Einträge werden am Anfang des Arrays platziert (`unshift`), um sie sofort sichtbar zu machen. Jeder Klick auf einen Link löst die Funktion `recordVisit` aus, die den entsprechenden Zähler im `visitCounts` -Objekt inkrementiert und sofort speichert. Dies triggered ein erneutes Rendering, um die dynamische Kategorie „Am häufigsten besucht“ zu aktualisieren. Diese Liste wird aus den gespeicherten Zählraten berechnet, nach Häufigkeit sortiert und auf die ersten 5 Einträge begrenzt.

Drag & Drop System

Die Anwendung nutzt die native HTML5 Drag & Drop API für die Sortierung. Beim Verschieben von Kacheln (`moveLink`) wird sichergestellt, dass dies nur innerhalb derselben Kategorie geschieht. Das Sortieren der Kategorien selbst erfolgt über einen separaten Dialog (`sortCategoriesDialog`). Hierbei wird mit einem temporären Array (`pendingCategoryOrder`) gearbeitet, sodass Änderungen erst wirksam werden, wenn der Nutzer den Dialog explizit mit „Speichern“ bestätigt. Globale Variablen tracken dabei das aktuell gezogene Element.

Import & Export

Die Export-Funktion erstellt ein JSON-Objekt, das einen Zeitstempel, die Kategorien-Reihenfolge und die gesamte Link-Liste enthält. Der Download wird clientseitig durch Erstellen eines temporären `<a>` -Tags und eines Blob-Objects ausgelöst. Beim Import wird die hochgeladene JSON-Datei gelesen und durch die Funktion `sanitizeImportedLinks` validiert.

Dabei werden Struktur und URLs geprüft (`new URL()`), und fehlende UUIDs werden neu generiert. Der Nutzer kann wählen, ob die importierten Daten die bestehenden ersetzen oder ergänzen sollen.

Automatischer Backup-Export

Ein Event-Listener auf dem `window`-Objekt überwacht das Schließen des Tabs (`beforeunload`). Wenn seit dem letzten Speichern Änderungen vorgenommen wurden (`hasChanges === true`), versucht das Skript automatisch, eine Backup-Datei herunterzuladen. Um doppelte Warnmeldungen des Browsers zu vermeiden, wird das Änderungs-Flag sofort zurückgesetzt. Sollte der Download vom Browser blockiert werden, dient eine generische Warnmeldung im `event.returnValue` als letztes Sicherheitsnetz, um den Nutzer auf den versuchten Export hinzuweisen.

4. DOM-Manipulation & Rendering

Das Rendering der Anwendung erfolgt vollständig dynamisch über JavaScript mittels `document.createElement`. Es wird keine Template-Engine verwendet.

Tile Factory

Die Funktion `createTile` erzeugt das DOM für eine einzelne Favoriten-Kachel. Ein entscheidendes Merkmal ist die Favicon-Logik: Um Datenschutz zu gewährleisten, wird versucht, das Icon direkt von der Ziel-Domain (`domain.com/favicon.ico`) zu laden. Schlägt dies fehl, wird kein externer Dienst angefragt, sondern ein lokal im Code eingebettetes SVG (Data URI) als Platzhalter verwendet. Die Interaktionselemente (Link, Bearbeiten-, Löschen-Buttons) werden ebenfalls dynamisch erstellt und mit Event-Listnern versehen. Drag-and-Drop-Handles werden nur für manuelle Kategorien eingefügt, nicht für die automatische „Am häufigsten besucht“-Liste.

Render-Pipeline

Die Hauptfunktion `render` leert den Container und baut die Ansicht neu auf. Zuerst wird die Liste der anzuzeigenden Links basierend auf der aktuellen Suchanfrage gefiltert (Suche in Titel, URL und Kategorie). Die verbleibenden Links werden nach Kategorien gruppiert. Sofern Treffer vorliegen, wird zunächst der Abschnitt „Am häufigsten besucht“ gerendert. Anschließend iteriert das Skript über die definierte Kategorien-Reihenfolge und erstellt für jede Kategorie einen Abschnitt mit den zugehörigen Kacheln. Abschließend wird die Vorschlagsliste für die Kategorie-Eingabe aktualisiert.

5. Sicherheit & Validierung

Die Anwendung setzt auf mehrere Sicherheitsmechanismen. Um Cross-Site-Scripting (XSS) zu verhindern, werden alle Nutzereingaben (Titel, URLs) ausschließlich über `textContent` in das DOM eingefügt, niemals über `innerHTML`. URLs werden beim Speichern und Importieren mit dem `new URL()` Constructor validiert, um malformed URLs abzufangen. Beim Import werden zudem nur Felder übernommen, die den erwarteten Datentypen entsprechen; ungültige Einträge werden verworfen. Hinsichtlich der Protokoll-Sicherheit erlaubt der Code explizit `http`, `https` und benutzerdefinierte Schemes. Eine explizite Blockliste für potenziell gefährliche Protokolle wie `javascript:` ist im aktuellen Code nicht implementiert, wobei sich die Anwendung hier auf das Sandboxing des Browsers und die Validierung durch den URL-Constructor verlässt.

6. Bekannte Einschränkungen

Trotz der umfassenden Funktionalität gibt es technische Grenzen. Die native HTML5 Drag & Drop API wird von mobilen Browsern (insbesondere auf iOS und Android) oft nicht oder nur eingeschränkt unterstützt, da Touch-Events nicht implementiert sind. Das Speicherlimit des `localStorage` ist browserabhängig (meist ca. 5-10 MB); bei einer extrem großen Anzahl von Links kann dieser Speicher erschöpft werden. Da es keine Backend-Komponente gibt, erfolgt keine automatische Synchronisation zwischen verschiedenen Geräten; der Nutzer muss den JSON-Export manuell übertragen, um Daten zu migrieren.

7. Datei-Struktur

Das Projekt besteht aus vier zentralen Dateien:

- **`index.html`** : Enthält das Grundgerüst der Seite, die Definition der Modal-Dialoge (`<dialog>`) und die Container-Elemente für das Rendering.
- **`styles.css`** : Beinhaltet das vollständige Styling, definiert CSS-Variablen für Light- und Dark-Themes und sorgt für ein responsives Design auf verschiedenen Bildschirmgrößen.
- **`app.js`** : Enthält die gesamte Anwendungslogik, inklusive Event-Listener, Storage-Management, Validierungsfunktionen und der Render-Pipeline.
- **`BEDIENUNGSANLEITUNG.md`** : Die Benutzerdokumentation mit Erklärungen zur Bedienung und Fehlerbehebung.

8. Erweiterungsmöglichkeiten

Für zukünftige Versionen bieten sich mehrere Erweiterungen an. Die Implementierung von Touch-Events (`touchstart` , `touchmove` , `touchend`) wäre notwendig, um Drag & Drop auch auf mobilen Geräten nutzbar zu machen. Eine Umwandlung in eine Progressive Web App (PWA) durch Hinzufügen eines `manifest.json` und eines Service Workers würde Offline-Nutzung und die Installation auf dem Homescreen ermöglichen. Zudem könnte eine optionale client-seitige Verschlüsselung des JSON-Exports mittels der Web Crypto API die Sicherheit sensibler Lesezeichen weiter erhöhen.